Vector Databases and ChromaDB Basics

Storing and searching embedded document chunks

Purpose

This short reference note is written for learning, semantic retrieval, and grounded question-answering experiments. It uses original explanatory content and is organized into coherent sections rather than unrelated filler.

1. Why a Vector Store?

After document chunks are embedded, the application needs a way to store vectors and retrieve similar ones efficiently. A vector store associates vectors with identifiers and often stores text documents and metadata alongside them. It supports nearest-neighbor search, and may support metadata filtering, persistence, and collection management. A vector store is one component of a retrieval system, not a complete question-answering application.

2. Collections, IDs, and Metadata

A collection groups related records. Each record should have a stable unique identifier, an embedding, and useful metadata. Depending on the setup, the original chunk text may be stored directly or referenced elsewhere. Metadata can include source file, page number, chunk index, and ingestion version. Clear IDs and metadata make it easier to update a document and trace search results back to the original PDF.

3. Similarity Search

At query time, the system embeds the question and requests the nearest vectors. The store returns candidates according to its configured distance metric and search behavior. The meaning of a returned distance or score depends on the metric and implementation. Do not assume that a particular numerical threshold works across models or datasets without testing it.

4. Persistence and Reproducibility

Persistent storage allows the collection to survive application restarts. Developers should know where the database files are stored and how collection names are configured. When the embedding model or chunking method changes, old records may no longer be compatible with new assumptions. A controlled re-ingestion process avoids a collection containing duplicate, stale, and incompatible vectors.

5. Filtering and Top-k

Top-k controls how many candidates are returned. A low value reduces prompt size but can miss useful evidence; a high value may include irrelevant passages and increase latency. Metadata filters can restrict search to a selected document or category. Tune these settings with representative queries and inspect the actual returned chunks, rather than choosing a value only because it is common in examples.

6. Operational Checks

Check collection counts after ingestion, inspect a few stored records, verify that embeddings have the expected dimension, and test queries with known answers. Confirm that source metadata survives retrieval. For updates, decide whether to replace records, delete old chunks, or rebuild a collection. Log errors and avoid silently swallowing failures, since a successful API response does not always prove that the intended corpus was indexed.

Key Takeaways

A vector database stores and retrieves vector representations of chunks. Good IDs, metadata, persistence, and versioned ingestion are essential for a maintainable document-search system.

End of document • Use source passages as evidence and verify claims during evaluation.